

DAY 2 : ASSIGNMENT 01-PYTHON ASSIGNMENT

Q1. L is a list defined as L= [11, 12, 13, 14].

A) WAP to add 50 and 60 to L.

CODE :

```
L = [11, 12, 13, 14]
```

```
L.append(50)
```

```
L.append(60)
```

```
print(L)
```

O/P:

```
[11, 12, 13, 14, 50, 60]
```

B) WAP to remove 11 and 13 from L.

Code :

```
L = [11, 12, 13, 14]
```

```
L.remove(11)
```

```
L.remove(13)
```

```
print(L)
```

O/P:

```
[12, 14]
```

C) WAP to sort L in ascending order

Code :

```
L = [11, 12, 13, 14]
```

```
L.sort()
```

```
print(L)
```

O/P:

```
[11, 12, 13, 14]
```

D) WAP to sort L in descending order.

Code :

```
L = [11, 12, 13, 14]
```

```
L.sort(reverse=True)
```

```
print(L)
```

O/P:

```
[14, 13, 12, 11]
```

E) WAP to search for 13 in L.

Code :

```
L = [11, 12, 13, 14]
```

```
if 13 in L:
```

```
    print("13 is present in L")
```

```
else:
```

```
    print("13 is not present in L")
```

O/P:

```
13 is present in L
```

F) WAP to count the number of elements present in L.

Code :

```
L = [11, 12, 13, 14]
```

```
count = len(L)
```

```
print("Number of elements in L:", count)
```

O/P:

```
Number of elements in L: 4
```

G) WAP to sum all the elements in L.

Code :

```
L = [11, 12, 13, 14]
```

```
sum_elements = sum(L)
```

```
print("Sum of all elements in L:", sum_elements) L = [11, 12, 13, 14]
```

O/P:

```
Sum of all elements in L: 50
```

H) WAP to sum all ODD numbers in L.

Code :

```
L = [11, 12, 13, 14]
```

```
sum_odd = sum(num for num in L if num % 2 != 0)
```

```
print("Sum of all odd numbers in L:", sum_odd)
```

O/P:

Sum of all odd numbers in L: 24

I) WAP to sum all EVEN numbers in L.

Code :

```
L = [11, 12, 13, 14]
```

```
sum_even = sum(num for num in L if num % 2 == 0)
```

```
print("Sum of all even numbers in L:", sum_even)
```

O/P:

Sum of all even numbers in L: 26

J) WAP to sum all PRIME numbers in L.

Code :

```
def is_prime(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    if n == 2:
```

```
        return True # 2 is prime
```

```
    if n % 2 == 0:
```

```
        return False # other even numbers are not prime
```

```
    for i in range(3, int(n**0.5) + 1, 2):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True
```

```
L = [11, 12, 13, 14]
```

```
sum_prime = sum(num for num in L if is_prime(num))
```

```
print("Sum of all prime numbers in L:", sum_prime)
```

O/P:

Sum of all prime numbers in L: 24

K) WAP to clear all the elements in L.

Code :

```
L = [11, 12, 13, 14]
```

```
L.clear()
```

```
print("L after clearing all elements:", L)
```

O/P:

L after clearing all elements: []

L) WAP to delete L.

Code :

```
L = [11, 12, 13, 14]
```

```
del L
```

```
print("L is deleted")
```

O/P:

L is deleted

Q2 . D is a dictionary defined as D= {1:5.6, 2:7.8, 3:6.6, 4:8.7, 5:7.7}.

(i) WAP to add new entry in D; key=8 and value is 8.8

Code :

```
D[8] = 8.8
```

```
print("Dictionary after adding new entry:", D)
```

O/P:

```
{1: 5.6, 2: 7.8, 3: 6.6, 4: 8.7, 5: 7.7, 8: 8.8}
```

(ii) WAP to remove key=2.

Code :

if 2 in D:

```
del D[2]
```

```
print("Dictionary after removing key=2:", D)
```

else:

```
print("Key=2 is not present in the dictionary.")
```

Output:

```
{1: 5.6, 3: 6.6, 4: 8.7, 5: 7.7, 8: 8.8}
```

(iii) WAP to check whether 6 key is present in D.

Code :

```
if 6 in D:
```

```
    print("Key=6 is present in the dictionary.")
```

```
else:
```

```
    print("Key=6 is not present in the dictionary.")
```

Output:

```
Key=6 is not present in the dictionary.
```

(iv) WAP to count the number of elements present in D.

Code :

```
num_elements = len(D)
```

```
print("Number of elements in the dictionary D:", num_elements)
```

Output:

```
Number of elements in the dictionary D: 5
```

(v) WAP to add all the values present D.

Code :

```
total_sum = sum(D.values())
```

```
print("Sum of all values in the dictionary D:", total_sum)
```

Output:

```
Sum of all values in the dictionary D: 36.6
```

(vi) WAP to update the value of 3 to 7.1.

Code :

```
if 3 in D:
```

```
    D[3] = 7.1
```

```
    print("Dictionary after updating key=3:", D)
```

```
else:
```

```
print("Key=3 is not present in the dictionary.")
```

Output:

```
{1: 5.6, 3: 7.1, 4: 8.7, 5: 7.7, 8: 8.8}
```

(vii) WAP to clear the dictionary

Code :

```
D.clear()
```

```
print("Dictionary D after clearing all elements:", D)
```

Output:

```
Dictionary D after clearing all elements: {}
```

Q3

S1 is a set defined as S1= [10, 20, 30, 40, 50, 60].

S2 is a set defined as S2= [40, 50, 60, 70, 80, 90].

A)WAP to add 55 and 66 in Set S1.

Code :

```
S1 = {10, 20, 30, 40, 50, 60}
```

```
# Adding elements to the set
```

```
S1.add(55)
```

```
S1.add(66)
```

```
print("Set S1 after adding 55 and 66:", S1)
```

O/P :

```
Set S1 after adding 55 and 66: {66, 10, 40, 50, 20, 55, 60, 30}
```

C) WAP to remove 10 and 30 from Set S1.

Code :

```
S1 = {10, 20, 30, 40, 50, 60}
```

```
# Removing elements from the set
```

```
S1.discard(10)
```

```
S1.discard(30)
```

```
print("Set S1 after removing 10 and 30:", S1)
```

O/P :

Set S1 after removing 10 and 30: {40, 50, 20, 60}

D) WAP to check whether 40 is present in S1.

Code :

```
S1 = {10, 20, 30, 40, 50, 60}
```

```
# Checking if an element is present
```

```
if 40 in S1:
```

```
    print("40 is present in S1")
```

```
else:
```

```
    print("40 is not present in S1")
```

O/P :

40 is present in S1

E) WAP to find the union between S1 and S2

Code :

```
S1 = {10, 20, 30, 40, 50, 60}
```

```
S2 = {40, 50, 60, 70, 80, 90}
```

```
# Union of two sets
```

```
union_set = S1.union(S2)
```

```
print("Union of S1 and S2:", union_set)
```

O/P :

Union of S1 and S2: {70, 40, 10, 80, 50, 20, 90, 60, 30}

F) WAP to find the intersection between S1 and S2.

Code :

```
S1 = {10, 20, 30, 40, 50, 60}
```

```
S2 = {40, 50, 60, 70, 80, 90}
```

```
# Intersection of two sets
```

```
intersection_set = S1.intersection(S2)
```

```
print("Intersection of S1 and S2:", intersection_set)
```

O/P :

Intersection of S1 and S2: {40, 50, 60}

G) WAP to find the S1 - S2.

Code :

```
S1 = {10, 20, 30, 40, 50, 60}
```

```
S2 = {40, 50, 60, 70, 80, 90}
```

```
# Set difference
```

```
difference_set = S1 - S2
```

```
print("Set difference (S1 - S2):", difference_set)
```

O/P :

Set difference (S1 - S2): {10, 20, 30}

Q4 . Write the following program.

- (i) WAP to print 100 random strings whose length between 6 and 8.
- (ii) WAP to print all prime numbers between 600 and 800.
- (iii) WAP to print all numbers between 100 and 1000 that are divisible by 7 and 9.

Code :

```
import random
```

```
import string
```

```
# Function to generate a random string of given length
```

```
def generate_random_string(length):
```

```
    letters = string.ascii_letters + string.digits + string.punctuation
```

```
    return "".join(random.choice(letters) for _ in range(length))
```

```
# Generate and print 100 random strings of lengths between 6 and 8
```

```
for _ in range(100):
```

```
    length = random.randint(6, 8)
```

```
    random_string = generate_random_string(length)
```

```
    print(random_string)
```

O/P :

A&B`0.Y

K;:}6,

&-j#*C.

+H84^e7

n[.\$x}:

jM7y:<t

xo0V&c3

`LdRq*i

L_TTmhG

NZY>1s,

B)

Function to check if a number is prime

def is_prime(num):

""" Function to check if a number is prime """

if num <= 1:

return False

if num == 2:

return True # 2 is prime

if num % 2 == 0:

return False # other even numbers are not prime

for i in range(3, int(num**0.5) + 1, 2):

if num % i == 0:

return False

return True

Print all prime numbers between 600 and 800

for num in range(600, 801):

if is_prime(num):

print(num)

O/P :

601

607

613

617

619

631

641

643

647

653

659

661

673

677

683

691

701

709

719

J727

733

739

743

751

757

761

769

773

787

797

C for num in range(100, 1001):

if num % 7 == 0 and num % 9 == 0:

print(num)

O/P :

126

189

252

315

378

441

504

567

630

693

756

819

882

945

Q5. WAP to create two lists of 10 random numbers between 10 and 30; Find

- (i) Common numbers in the two lists
- (ii) Unique numbers in both the list
- (iii) Minimum in both the list
- (iv) Maximum in both the list
- Sum of both the lists

Code :

```
import random

# Generate two lists of 10 random numbers between 10 and 30
list1 = [random.randint(10, 30) for _ in range(10)]
list2 = [random.randint(10, 30) for _ in range(10)]

# (i) Common numbers in the two lists
common_numbers = list(set(list1) & set(list2))

# (ii) Unique numbers in both lists
unique_list1 = list(set(list1) - set(list2))
unique_list2 = list(set(list2) - set(list1))

# (iii) Minimum in both lists
min_list1 = min(list1)
```

```
min_list2 = min(list2)

# (iv) Maximum in both lists
max_list1 = max(list1)
max_list2 = max(list2)

# Sum of both lists
sum_list1 = sum(list1)
sum_list2 = sum(list2)
total_sum = sum_list1 + sum_list2

# Print results
print("List 1:", list1)
print("List 2:", list2)
print("Common numbers:", common_numbers)
print("Unique numbers in List 1:", unique_list1)
print("Unique numbers in List 2:", unique_list2)
print("Minimum in List 1:", min_list1)
print("Minimum in List 2:", min_list2)
print("Maximum in List 1:", max_list1)
print("Maximum in List 2:", max_list2)
print("Sum of List 1:", sum_list1)
print("Sum of List 2:", sum_list2)
print("Total sum of both lists:", total_sum)
```

O/P:

List 1: [23, 26, 15, 24, 29, 27, 29, 21, 14, 12]

List 2: [14, 28, 20, 13, 17, 22, 21, 29, 17, 13]

Common numbers: [21, 29, 14]

Unique numbers in List 1: [12, 15, 23, 24, 26, 27]

Unique numbers in List 2: [20, 22, 28, 13, 17]

Minimum in List 1: 12

Minimum in List 2: 13

Maximum in List 1: 29

Maximum in List 2: 29

Sum of List 1: 210

Sum of List 2: 207

Total sum of both lists: 417

Q6. WAP to create a list of 100 random numbers between 100 and 900. Count and print the:

A) All odd numbers

B) All even numbers

C) All prime numbers

Code :

```
import random
```

```
# Function to check if a number is prime
```

```
def is_prime(num):
```

```
    """ Function to check if a number is prime """
```

```
    if num <= 1:
```

```
        return False
```

```
    if num == 2:
```

```
        return True # 2 is prime
```

```
    if num % 2 == 0:
```

```
        return False # other even numbers are not prime
```

```
    for i in range(3, int(num**0.5) + 1, 2):
```

```
        if num % i == 0:
```

```
            return False
```

```
    return True
```

```
# Generate a list of 100 random numbers between 100 and 900
```

```
random_numbers = [random.randint(100, 900) for _ in range(100)]
```

```
# Initialize counts
```

```
odd_count = 0
```

```
even_count = 0
```

```

prime_numbers = []
# Classify numbers
for num in random_numbers:
    if num % 2 != 0: # Odd number
        odd_count += 1
    else: # Even number
        even_count += 1

    if is_prime(num): # Prime number
        prime_numbers.append(num)
# Print results
print(f"All odd numbers: {odd_count}")
print(f"All even numbers: {even_count}")
print(f"All prime numbers: {prime_numbers}")

```

O/P:

All odd numbers: 53

All even numbers: 47

All prime numbers: [101, 103, 107, 109, 113, 127, 131, 137, 139, 149, ...]

Q7. D is a dictionary defined as D={1:"One",2:"Two",3:"Three",4:"Four", 5:"Five"}. WAP to read all the keys and values from dictionary and write to the file in the given below format. Key1, Value1 Key2, Value2 Key3, Value3

Code :

```

# Define the dictionary
D = {1: "One", 2: "Two", 3: "Three", 4: "Four", 5: "Five"}
# Open a file to write the results
file_name = "dictionary_keys_values.txt"
with open(file_name, 'w') as file:
    for key, value in D.items():
        file.write(f"Key{key}, {value} ")

```

```
print(f"Keys and values from dictionary have been written to '{file_name}'.")
```

O/P :

Keys and values from dictionary have been written to 'dictionary_keys_values.txt'.

Q8. L is a list defined as L={"One","Two","Three","Four","Five"}. WAP to count the length of each element from a list and write to the file in the given below format: One, 3 Two, 3 Four, 4

Code :

```
# Define the list
```

```
L = ["One", "Two", "Three", "Four", "Five"]
```

```
# Open a file to write the results
```

```
file_name = "element_lengths.txt"
```

```
with open(file_name, 'w') as file:
```

```
    for item in L:
```

```
        length = len(item) # Calculate the length of the current item
```

```
        file.write(f"{item}, {length}\n") # Write to the file in the specified format
```

```
print(f"Element lengths have been written to '{file_name}'.")
```

O/P:

One, 3

Two, 3

Three, 5

Four, 4

Five, 4

Element lengths have been written to 'element_lengths.txt'.

Q9. Write to the file 100 random strings whose length between 10 and 15.

Code :

```
import random
```

```
import string
```

```
# Function to generate a random string of given length
```

```

def generate_random_string(length):
    letters = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(letters) for _ in range(length))

# Define the number of strings to generate
num_strings = 100

# Open a file to write random strings
file_name = "random_strings.txt"

with open(file_name, 'w') as file:
    for _ in range(num_strings):
        # Generate random length between 10 and 15
        length = random.randint(10, 15)

        # Generate random string of specified length
        random_string = generate_random_string(length)

        # Write the random string to the file
        file.write(f"{random_string}\n")

print(f"{num_strings} random strings have been written to '{file_name}'.")

```

O/P:

100 random strings have been written to 'random_strings.txt'.

Q10. Write to the file all prime numbers between 600 and 800.

Code :

```

def is_prime(num):
    """ Function to check if a number is prime """

```

```

if num <= 1:
    return False
if num == 2:
    return True # 2 is prime
if num % 2 == 0:
    return False # other even numbers are not prime
for i in range(3, int(num**0.5) + 1, 2):
    if num % i == 0:
        return False
return True

```

Define the range

start = 600

end = 800

Open a file to write prime numbers

file_name = "prime_numbers_600_to_800.txt"

with open(file_name, 'w') as file:

for num in range(start, end + 1):

if is_prime(num):

file.write(f"{num}\n")

print(f"Prime numbers between {start} and {end} have been written to '{file_name}'.")

O/P :

Prime numbers between 600 and 800 have been written to
'prime_numbers_600_to_800.txt'.

Q11. WAP to calculate the time taken by a program.

Code :

import time

```

def your_program():
    # Replace this with the actual code you want to measure
    for i in range(5):
        print(i)
        time.sleep(1) # Simulating some computation
start_time = time.time()
your_program() # Call your program here
end_time = time.time()
execution_time = end_time - start_time
print(f"Execution time: {execution_time:.6f} seconds")

```

O/P :

```

0
1
2
3
4
Execution time: 5.007922 seconds

```

Q12. WAP to sort following number of elements in a list, calculate time taken and plot the graph. Number of elements in list Time Taken 5k T1 10k T2 15k T3 20k T4 25k T5

Code :

```

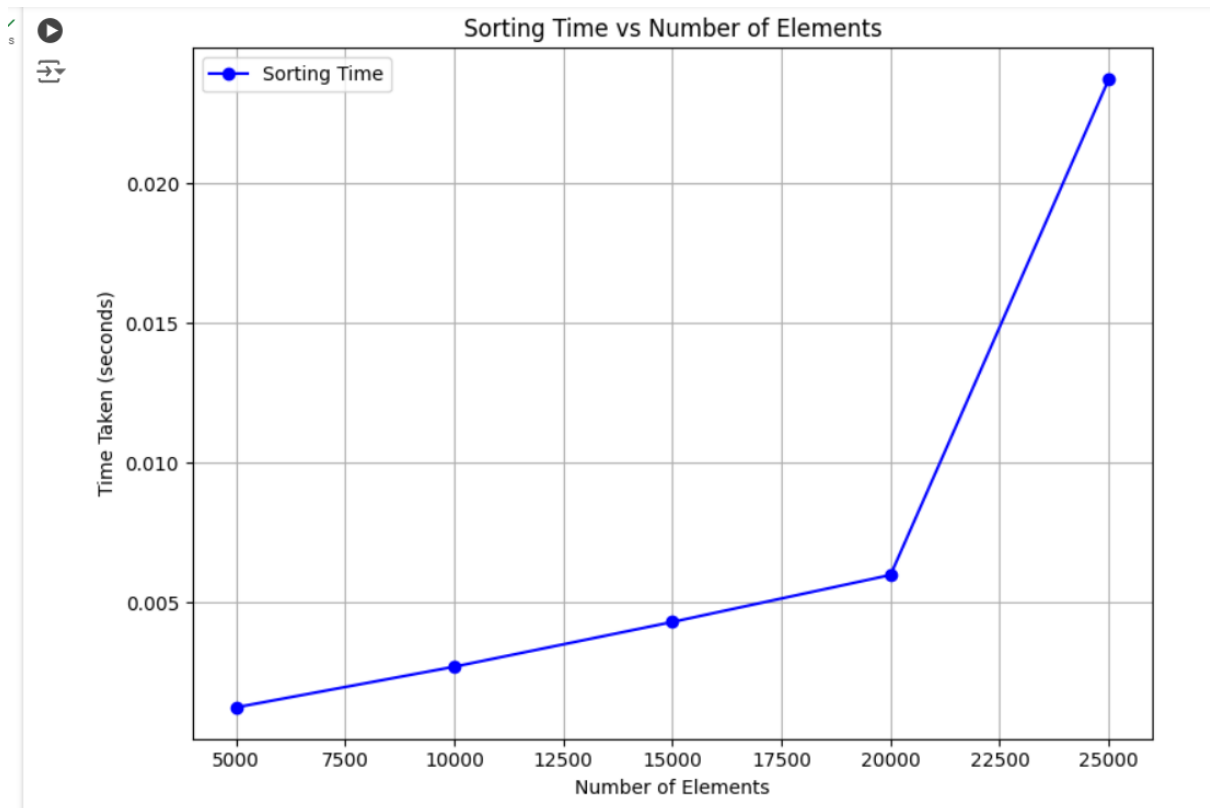
import time
import random
import matplotlib.pyplot as plt
def sort_and_measure_time(n):
    data = [random.randint(1, 1000) for _ in range(n)]
    start_time = time.time()
    data_sorted = sorted(data)
    end_time = time.time()
    time_taken = end_time - start_time
    return time_taken
def main():
    sizes = [5000, 10000, 15000, 20000, 25000]

```

```
time_taken = []  
for size in sizes:  
    t = sort_and_measure_time(size)  
    time_taken.append(t)  
    print(f"Time taken for {size} elements: {t:.6f} seconds")  
    plt.figure(figsize=(8, 6))  
    plt.plot(sizes, time_taken, marker='o', linestyle='-', color='b', label='Sorting Time')  
    plt.title('Sorting Time vs Number of Elements')  
    plt.xlabel('Number of Elements')  
    plt.ylabel('Time Taken (seconds)')  
    plt.grid(True)  
    plt.legend()  
    plt.tight_layout()  
    plt.show()  
  
if __name__ == "__main__":  
    main()
```

O/P:

```
Time taken for 5000 elements: 0.001258 seconds  
Time taken for 10000 elements: 0.002712 seconds  
Time taken for 15000 elements: 0.004312 seconds  
Time taken for 20000 elements: 0.005994 seconds  
Time taken for 25000 elements: 0.023721 seconds
```



Q13. WAP to create a dictionary of student marks in five subjects and you have to find the student having maximum and minimum average marks.

Code :

```
student_marks = {
    'Alice': [85, 90, 88, 92, 89],
    'Bob': [78, 82, 80, 75, 79],
    'Charlie': [92, 88, 94, 90, 87],
    'David': [68, 72, 70, 65, 69],
    'Eve': [95, 98, 96, 92, 97]
}

average_marks = {}

for student, marks in student_marks.items():
    average = sum(marks) / len(marks)
    average_marks[student] = average

max_student = max(average_marks, key=average_marks.get)
```

```
max_average = average_marks[max_student]
min_student = min(average_marks, key=average_marks.get)
min_average = average_marks[min_student]
print("Student with maximum average marks:")
print(f"{max_student}: {max_average}")
```

```
print("\nStudent with minimum average marks:")
print(f"{min_student}: {min_average}")
```

O/P :

Student with maximum average marks:

Eve: 95.6

Student with minimum average marks:

David: 68.8